Bài 6

PHP hướng đối tượng, Composer và kiến trúc MVC

Bài 6

PHP hướng đối tượng, Composer và kiến trúc MVC

Mục tiêu và sản phẩm cuối bài

- Hiểu class, object, constructor, visibility và dependency ở mức nhập môn.
- Biết vai trò của Composer và PSR-4 autoloading.
- Phân biệt trách nhiệm Model/Repository, View, Controller và Router.
- Refactor CRUD đã làm sang cấu trúc MVC tối giản.
- Áp dụng ví dụ demo vào products/orders của MiniShop.

Checkpoint: một module CRUD được refactor sang cấu trúc MVC: Controller gọi Repository, View chỉ hiển thị.

6.1 Class và object

Class là bản thiết kế; object là đối tượng cụ thể được tạo từ class.

Property lưu trạng thái; method mô tả hành vi.

public, private, protected kiểm soát khả năng truy cập.

Constructor khởi tạo dependency cần thiết, ví dụ PDO.

OOP giúp gom dữ liệu và hành vi liên quan vào một đơn vị rõ ràng.

Bài 6

Demo class: ProductRepository

MiniShop

```
class ProductRepository {
  public function __construct(private PDO $pdo) {}

  public function findAll(): array {
    $stmt = $this->pdo->query(
      'SELECT id, code, name, price, stock FROM products ORDER BY code'
    );
    return $stmt->fetchAll();
  }

  public function findById(int $id): ?array {
    $stmt = $this->pdo->prepare(
      'SELECT * FROM products WHERE id = :id'
    );
    $stmt->execute(['id' => $id]);
    return $stmt->fetch() ?: null;
  }
}
```

6.2 Dependency injection

Dependency là thứ một class cần để làm việc, ví dụ repository cần PDO.

Thay vì tạo PDO trực tiếp khắp nơi, truyền PDO vào constructor.

Cách này giúp mã dễ kiểm thử và dễ thay cấu hình.

Class không nên tự biết quá nhiều về cách tạo dependency.

Sau này framework sẽ tự động hóa việc này.

6.3 Composer và PSR-4 autoloading

Composer quản lý thư viện và autoload class trong project PHP.

PSR-4 ánh xạ namespace với thư mục mã nguồn.

Sau khi cấu hình autoload, có thể dùng class mà không require thủ công từng file.

Cấu trúc thường dùng: src/Controller, src/Repository, src/Service.

Chạy composer dump-autoload khi thay đổi cấu hình autoload.

Demo Composer: autoload tối giản

```
{
  "autoload": {
    "psr-4": {
      "App\": "src/"
    }
  }
}

// public/index.php
require __DIR__ . '/../vendor/autoload.php';
```

6.4 MVC và trách nhiệm từng tầng

Repository/Model: xử lý dữ liệu và truy vấn CSDL.

View: hiển thị giao diện; hạn chế logic nghiệp vụ.

Controller: nhận request, kiểm tra dữ liệu, gọi repository/service và chọn view/redirect.

Router: ánh xạ URL/action tới controller tương ứng.

MVC giúp phân vai rõ để bảo trì dễ hơn, không tự động làm mã tốt hơn.

6.5 Front Controller và router tối giản

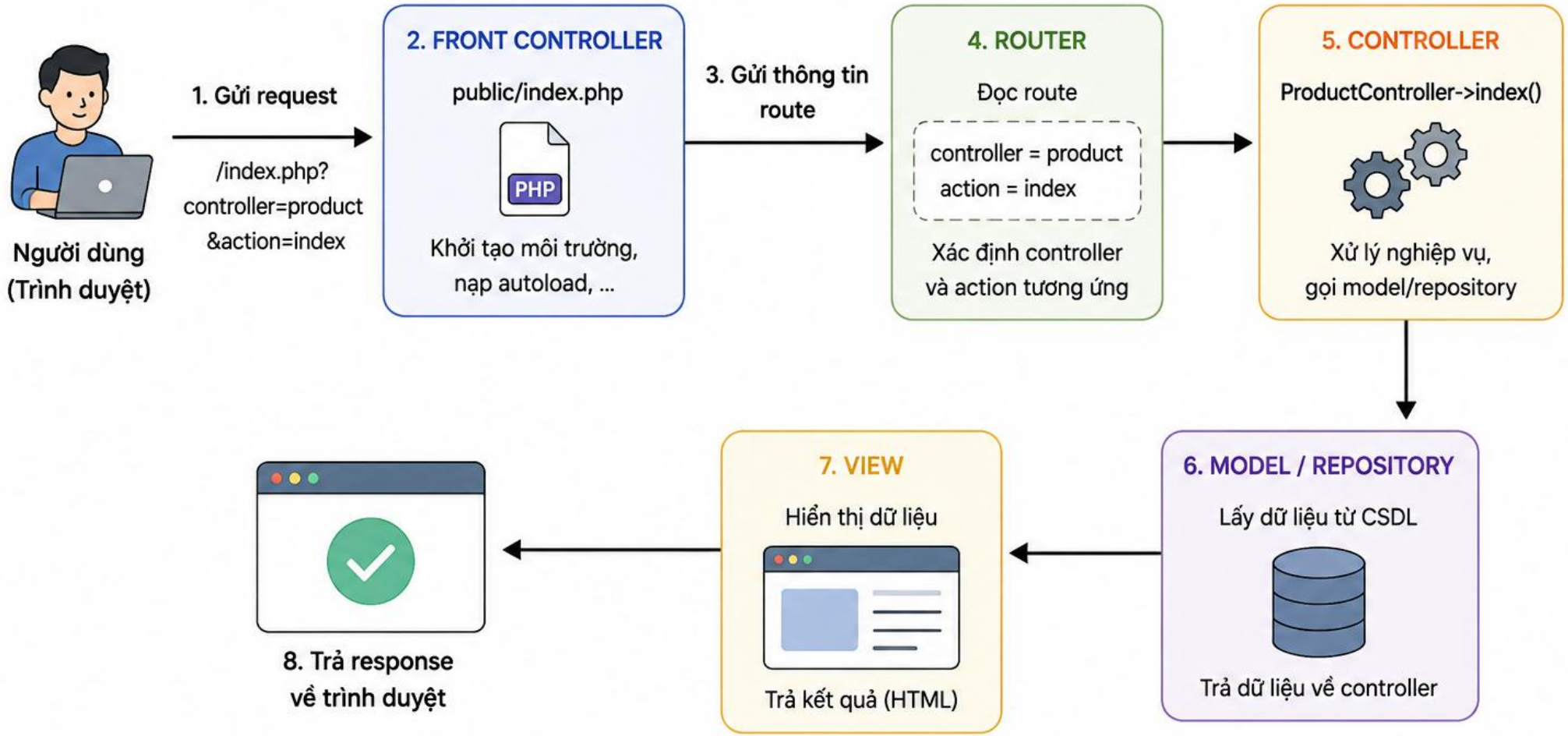
Front controller là một file vào chính, thường public/index.php.

Mọi request đi qua front controller để khởi tạo môi trường và điều hướng.

Router đọc route/action và gọi controller tương ứng.

Có thể dùng query string: ?controller=product&action=index.

LUỒNG XỬ LÝ REQUEST (FRONT CONTROLLER)



Tóm tắt: Tất cả request đi vào public/index.php (Front Controller) ⟶ Router xác định Controller + Action ⟶ Controller xử lý, gọi dữ liệu ⟶ View hiển thị ⟶ Trả kết quả về trình duyệt.

Bài 6

Bài tập nhanh: phân tầng mã

Quan sát CRUD từ Bài 5

Bài tập

Trong chức năng thêm sản phẩm, đoạn validate thuộc tầng nào? INSERT thuộc tầng nào? HTML form thuộc tầng nào?

Đáp án

- Validate: Controller hoặc Form Request đơn giản.
- INSERT: Repository/Model.
- HTML form: View.

Demo Controller: danh sách sản phẩm

MiniShop

```
class ProductController {
  public function __construct(private ProductRepository
$products) {}

  public function index(): void {
    $products = $this->products->findAll();
    require __DIR__ . '/../../views/products/index.php';
  }

  public function create(): void {
    require __DIR__ . '/../../views/products/create.php';
  }
}
```

Bài 6 — Bước 1: Chuẩn bị project MVC

Mục tiêu: sinh viên nhìn thấy project không còn là vài file PHP rời rạc



Cấu trúc thư mục

```
minishop_mvc_demo/
  ├── config/config.php       # cấu hình CSDL
  ├── database/minishop.sql   # schema + dữ liệu mẫu
  ├── public/index.php        # điểm vào duy nhất
  ├── src/Core/               # Auth, Database, Controller
  ├── src/Controller/         # nhận request
  ├── src/Repository/         # SQL/PDO
  └── src/View/               # giao diện
```

Checkpoint

- Import CSDL không lỗi, có 5 bảng chính.
- composer dump-autoload tạo thư mục vendor.
- Truy cập public/index.php hiển thị danh sách sản phẩm.
- Không sửa trực tiếp file trong vendor/.

Bài 6 — Bước 2: Composer autoload và kết nối PDO

Mục tiêu: dùng namespace thay cho require thủ công; PDO được tạo tập trung một lần

1 composer.json ánh xạ namespace App\ → src/

```
{
  "autoload": {
    "psr-4": { "App\\": "src/" }
  }
}
```

2 public/index.php nạp autoload và helpers

```
require dirname(__DIR__) . '/vendor/autoload.php';
require dirname(__DIR__) . '/src/Core/helpers.php';
```

3 Database::connection() tạo PDO chuẩn utf8mb4

```
$dsn =
'mysql:host=localhost;dbname=minishop_mvc;charset=utf8mb4';
new PDO($dsn, 'root', '', [
  PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
```

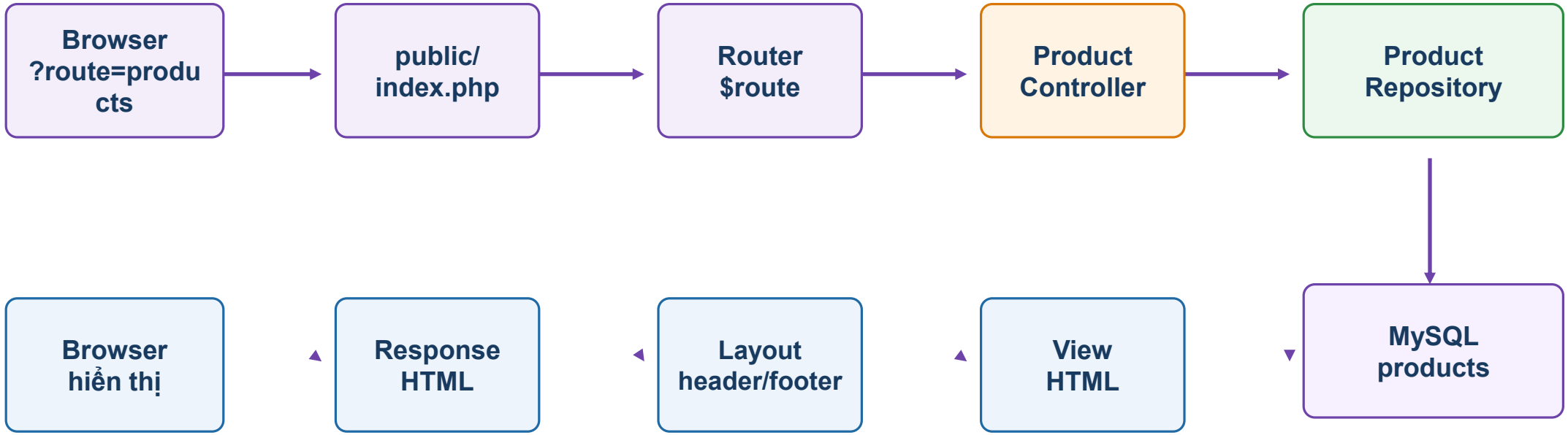
4 Repository nhận PDO qua constructor

```
$pdo = Database::connection();
$productRepository = new ProductRepository($pdo);
$productController = new
ProductController($productRepository);
```

Controller không tự tạo PDO; Repository nhận PDO để truy vấn dữ liệu. Đây là Dependency Injection ở mức cơ bản.

Bài 6 — Bước 3: Luồng request qua Front Controller

Mục tiêu: sinh viên đọc được đường đi của một request trong MVC



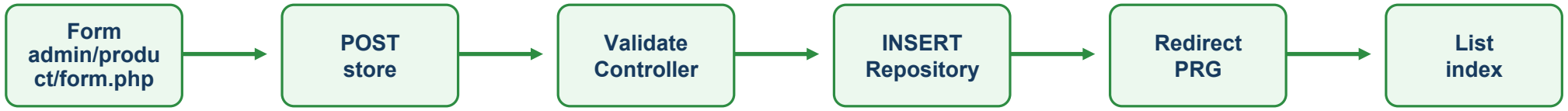
Tầng	Không nên làm	Nên làm
Controller	Viết SQL dài	Nhận request, validate, gọi Repository
Repository	In HTML	Chứa SQL/PDO, trả array dữ liệu
View	Truy vấn CSDL	Hiển thị biến đã được Controller truyền vào

Bài 6 — Bước 4: Refactor CRUD sản phẩm sang MVC

Mục tiêu: một chức năng CRUD được bóc tách thành Route → Controller → Repository → View

Chức năng	Route	Controller	Repository	View
Danh sách	GET ?route=products	ProductController@index	ProductRepository@findAll	product/index.php
Form thêm	GET ?route=admin/products/create	AdminProductController@createForm	categories()	admin/product/form.php
Lưu mới	POST ?route=admin/products/store	store() + validate + CSRF	create($data)	redirect về danh sách
Sửa	GET/POST edit/update	editForm()/update()	find($id), update()	form.php
Xóa	POST delete	delete() + CSRF	delete($id)	redirect

Ví dụ luồng thêm sản phẩm



Checkpoint: sau khi refactor, View không còn SQL; mọi INSERT/UPDATE/DELETE đi qua Controller và Repository.

Bài 6

Thực hành trên sản phẩm: refactor CRUD sang MVC

Từ module đã làm ở Bài 5

Bài tập

Chuyển CRUD products hoặc một bảng tương đương trong đề tài nhóm thành: Controller + Repository + View.

- View không chứa SQL.
- Controller không chứa HTML dài.
- Repository chịu trách nhiệm truy vấn.
- Route chạy được qua public/index.php.

Bài tập về nhà 6: tiếp nối từ buổi trước

Cá nhân	Nhóm	Yêu cầu nộp
• Refactor CRUD cá nhân sang class Repository. • Tạo tối thiểu một Controller và một View tách riêng. • Ghi lại trước/sau khi refactor trong README.	• Thống nhất cấu trúc thư mục nhóm: public, src, views, config, database. • Chuyển một chức năng chính của bài tập lớn sang MVC. • Cập nhật route dự kiến cho các chức năng còn lại. • Mỗi thành viên phụ trách ít nhất một file/class.	• Link repository. • Ảnh cấu trúc thư mục. • README có mô tả route và phân công. • Commit của từng thành viên.

Bài 7

Xác thực, phân quyền và bảo mật ứng dụng

Bài 7

Xác thực, phân quyền và bảo mật ứng dụng

Mục tiêu và sản phẩm cuối bài

- Phân biệt authentication và authorization.
- Lưu mật khẩu bằng password_hash và kiểm tra bằng password_verify.
- Quản lý session, cookie và logout an toàn ở mức nhập môn.
- Kiểm tra role và ownership ở server, không chỉ ẩn nút trên giao diện.
- Thêm CSRF token cho form thay đổi dữ liệu (tránh giả mạo yêu cầu)

Checkpoint: MiniShop có đăng nhập, role admin/staff/customer và guard cho quản trị sản phẩm, xem đơn hàng.

Bài 7

7.1 Authentication và authorization

Hai câu hỏi khác nhau

- Authentication: người dùng là ai? Ví dụ đăng nhập bằng email/mật khẩu.
- Authorization: người dùng được làm gì? Ví dụ admin mới được quản lý sản phẩm.
- Hai bước này khác nhau và đều phải kiểm tra ở server.
- MiniShop dùng role trong users: admin, staff, customer.
- Không cho truy cập trực tiếp URL quản trị khi chưa đủ quyền.

7.2 Lưu mật khẩu đúng cách

Không lưu mật khẩu rõ trong CSDL.

Không dùng MD5/SHA1 tự viết cho mật khẩu.

Dùng password_hash() khi tạo tài khoản.

Dùng password_verify() khi đăng nhập.

Cột password_hash cần VARCHAR(255).

Demo đăng nhập: users và password_hash

```
$stmt = db()->prepare('SELECT * FROM users WHERE email = :email AND
status = "active"');
$stmt->execute(['email' => $email]);
$user = $stmt->fetch();

if ($user && password_verify($password, $user['password_hash'])) {
  session_regenerate_id(true);
  $_SESSION['user_id'] = $user['id'];
  $_SESSION['role'] = $user['role'];
  header('Location: index.php');
  exit;
}

$error = 'Email hoặc mật khẩu không đúng.';
```

7.3 Session và cookie

Session lưu trạng thái đăng nhập phía server.

Cookie thường chỉ giữ session id phía trình duyệt.

Sau đăng nhập thành công nên gọi session_regenerate_id(true).

Logout cần xóa dữ liệu session và chuyển về trang đăng nhập.

Không lưu role hoặc user_id vào form ẩn để tin tưởng ở server.

Bài 7

7.4 Guard quyền ở server

Không chỉ ẩn nút trên giao diện

- Mỗi route quan trọng cần kiểm tra đăng nhập và role.
- Admin: quản lý danh mục, sản phẩm, tài khoản, trạng thái đơn hàng.
- Staff: xem và xử lý đơn hàng được phân công.
- Customer: xem sản phẩm, tạo đơn, xem/hủy đơn của chính mình.
- Ẩn nút trên giao diện là chưa đủ; phải chặn request trực tiếp.

Bài 7

Demo guard: chỉ admin vào trang quản trị

MiniShop

```
function requireRole(array $roles): void {
  if (!isset($_SESSION['user_id'])) {
    header('Location: login.php'); exit;
  }
  if (!in_array($_SESSION['role'], $roles, true)) {
    http_response_code(403);
    echo 'Bạn không có quyền truy cập.'; exit;
  }
}

// Trang quản trị sản phẩm
requireRole(['admin']);
```

Bài 7

7.5 Ownership: quyền sở hữu dữ liệu

Không tin id truyền từ URL

- Ownership nghĩa là người dùng chỉ thao tác dữ liệu thuộc quyền của mình.
- Khách hàng chỉ được xem hoặc hủy đơn hàng của chính mình.
- Nhân viên chỉ xử lý đơn hàng thuộc phạm vi được phân công nếu hệ thống có phân công.
- Không tin id truyền từ URL nếu chưa đối chiếu với session.
- Đây là lỗi phổ biến trong bài tập web nhập môn.

7.6 CSRF- Cross-Site Request Forgery (Giả mạo yêu cầu liên trang)

CSRF là tình huống người dùng đang đăng nhập bị dụ gửi request ngoài ý muốn.

Các form thay đổi dữ liệu nên có CSRF token.

Token sinh ra trong session và gửi kèm form.

Server kiểm tra token trước khi INSERT/UPDATE/DELETE/hủy đăng ký.

Sai token thì từ chối thao tác.

Demo CSRF token cho form đăng ký/hủy

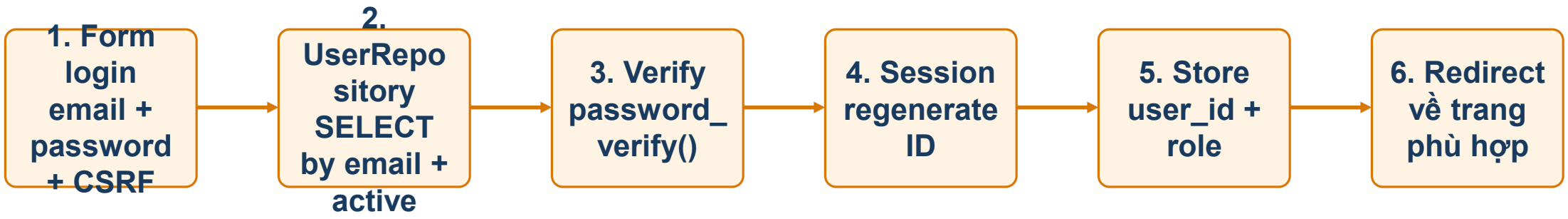
```
// tạo token
$_SESSION['csrf'] ??= bin2hex(random_bytes(32));

// trong form
<input type="hidden" name="csrf" value="<?=
e($_SESSION['csrf']) ?>">

// khi xử lý POST
if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ??
'')) {
  http_response_code(403);
  exit('CSRF token không hợp lệ.');
}
```

Bài 7 — Bước 5: Thêm đăng nhập bằng users + password_hash

Mục tiêu: phân biệt xác thực người dùng với phân quyền chức năng



Code lõi cần giải thích

```
$user = $users->findActiveByEmail($email);

if (!$user || !password_verify($password, $user['password_hash'])) {
    flash('error', 'Email hoặc mật khẩu không đúng.');
    redirect('login');
}

Auth::login($user);
```

Bảng users cần tối thiểu

field	ý nghĩa
email	định danh đăng nhập, UNIQUE
password_hash	kết quả password_hash(), không phải mật khẩu rõ
role	customer/admin
status	active/locked

đăng nhập sai → thông báo lỗi; đăng nhập đúng → session có user_id và role; logout xóa session.

Bài 7 — Bước 6: Role guard, ownership và CSRF

Mục tiêu: không chỉ ẩn nút trên giao diện; server phải tự bảo vệ thao tác

Ba lớp bảo vệ cần có trong MiniShop

1. **Login guard** — chưa đăng nhập → chuyển login
2. **Role guard** — customer không vào admin/products
3. **Ownership** — customer chỉ xem đơn của chính mình
4. **CSRF** — POST phải kèm token từ session

Route	Ai được phép?	Kiểm tra ở đâu?
GET products	mọi người	không cần login
GET my-orders	customer	Auth::requireRole
GET admin/products	admin	Auth::requireRole
POST delete product	admin + CSRF	Controller

```
Auth::requireRole('admin');
Auth::verifyCsrf($_POST['csrf'] ?? null);

$id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT);
$products->delete($id);
```

Đăng nhập customer → nhập URL admin/products → 403; xóa CSRF token khi delete → 419.

Bài 7

Thực hành trên sản phẩm: bảo vệ luồng đặt hàng

Áp dụng vào MiniShop hoặc đề tài nhóm

Bài tập tại slide

Thêm kiểm tra: đã đăng nhập, role=customer, sản phẩm còn hàng, CSRF hợp lệ và chỉ hủy đơn của chính mình.

- Người chưa đăng nhập bị chuyển login.
- Khách không thể hủy đơn của người khác.
- CSRF sai bị từ chối.

Bài tập về nhà 7: tiếp nối từ buổi trước

Cá nhân	Nhóm	Yêu cầu nộp
• Hoàn thiện đăng nhập/đăng xuất trong repository cá nhân. • Thêm password_hash/password_verify cho tài khoản mẫu. • Viết test case truy cập URL khi chưa đăng nhập.	• Tích hợp phân quyền vào project nhóm. • Ít nhất một route admin và một route student được guard ở server. • Thêm CSRF token cho form thay đổi dữ liệu. • Cập nhật README: tài khoản test và ma trận quyền.	• Link repository. • Danh sách tài khoản test. • Ảnh minh họa trường hợp bị chặn quyền. • Commit từng thành viên.

Bài 8

Fetch API, JSON, kiểm thử và triển khai

Bài 8

Fetch API, JSON, kiểm thử và triển khai

- Hiểu endpoint JSON và phân biệt với trang HTML.
- Dùng Fetch API, Promise và async/await ở mức nhập môn.
- Cập nhật DOM an toàn từ dữ liệu JSON.
- Kiểm thử form, quyền, nghiệp vụ và lỗi phổ biến.
- Chuẩn bị bản release candidate cho bài tập lớn.

Checkpoint: live search sản phẩm và endpoint kiểm tra tồn kho hoạt động không tải lại trang.

Bài 8

JSON trong ứng dụng Web

JSON (JavaScript Object Notation) là định dạng văn bản dùng để trao đổi dữ liệu giữa client và server.

- dữ liệu gọn, dễ đọc và dễ xử lý;
- phù hợp cho Fetch API/AJAX;
- giúp **tách dữ liệu khỏi giao diện HTML**;
- là định dạng phổ biến khi xây dựng **Web API/REST API**.

```
{
  "id": 4,
  "code": "USB-32",
  "name": "USB 32GB",
  "price": 150000
}
```

HTML dùng để trình bày; JSON chủ yếu dùng để trao đổi dữ liệu.

Bài 8

8.1 Endpoint JSON

Trả dữ liệu cho JavaScript

- Endpoint JSON trả dữ liệu cho JavaScript thay vì trả trang HTML hoàn chỉnh.
- Đặt header Content-Type: application/json.
- Dữ liệu trả về cần tối giản và đúng quyền người dùng.
- Không trả password_hash hoặc dữ liệu nhạy cảm ra JSON.
- Trong MiniShop: endpoint tìm sản phẩm và endpoint kiểm tra tồn kho.

```
header('Content-Type: application/json;
charset=utf-8');

$keyword = trim($_GET['q'] ?? '');
$data = $products->searchActive($keyword);

echo json_encode([
    'ok'   => true,
    'data' => $data
], JSON_UNESCAPED_UNICODE);
```

ApiProductController, server khai báo JSON và lấy dữ liệu từ Repository

```
header('Content-Type: application/json; charset=utf-8');
$keyword = trim($_GET['q'] ?? '');
$data = $products->searchActive($keyword);
echo json_encode([
    'ok'   => true,
    'data' => $data
], JSON_UNESCAPED_UNICODE);
```

JavaScript gọi endpoint bằng Fetch API:

```
const res = await fetch(
'index.php?route=api/products&q=' +
encodeURIComponent(keyword), { headers: { 'Accept':
'application/json' } } ); const json = await res.json();
```

Sau đó dữ liệu JSON được dùng để cập nhật bảng sản phẩm mà **không tải lại toàn bộ trang**.

Người dùng gõ "USB"
↓
Fetch API
↓
?route=api/products&q=USB
↓
ApiProductController
↓
ProductRepository::searchActive()
↓
MySQL
↓
JSON { ok, data }
↓
JavaScript cập nhật bảng

Bài 8

Demo endpoint JSON: tìm sản phẩm

MiniShop

```
header('Content-Type: application/json; charset=utf-8');

$keyword = trim($_GET['q'] ?? '');
$products = $productRepository->searchActiveProducts($keyword);

echo json_encode([
  'data' => $products,
  'count' => count($products),
], JSON_UNESCAPED_UNICODE);
```

8.2 Fetch API và async/await

fetch() gửi HTTP request từ trình duyệt.

await response.json() chuyển JSON thành object/array trong JavaScript.

Cần kiểm tra response.ok để xử lý lỗi HTTP.

Khi người dùng nhập từ khóa, gửi request và render kết quả.

Bài 8

Demo Fetch: gọi endpoint tìm sản phẩm

async/await

```
async function searchProducts(keyword) {
  const res = await fetch('/api/products.php?q=' +
encodeURIComponent(keyword));
  if (!res.ok) {
    throw new Error('Không tải được dữ liệu sản phẩm');
  }
  const json = await res.json();
  renderProducts(json.data);
}
```

8.3 Cập nhật DOM an toàn

- DOM là cây phần tử HTML hiện trên trình duyệt.
- Có thể tạo phần tử bằng document.createElement thay vì ghép chuỗi HTML dài.
- Khi chèn dữ liệu người dùng, ưu tiên textContent để tránh XSS.
- Giao diện cần có trạng thái loading, rỗng và lỗi.
- Kết quả Fetch phải thống nhất với dữ liệu từ CSDL.

Fetch lấy dữ liệu JSON; DOM đưa dữ liệu đó lên giao diện; cập nhật DOM an toàn giúp dữ liệu chỉ được hiển thị, không vô tình trở thành mã thực thi.

8.4 Debounce và trạng thái giao diện

- Không nên gửi request sau mỗi ký tự ngay lập tức.
- Debounce đợi người dùng dừng gõ một khoảng ngắn rồi mới gọi API.
- Trạng thái loading giúp người dùng biết hệ thống đang xử lý.
- Khi không có kết quả, cần hiển thị thông báo rõ.
- Khi API lỗi, không để giao diện im lặng.

8.5 Kiểm thử theo lớp

Từ dữ liệu đến API

- Kiểm thử dữ liệu: thiếu trường, sai định dạng, ký tự đặc biệt.
- Kiểm thử quyền: chưa đăng nhập, sai role, truy cập URL trực tiếp.
- Kiểm thử nghiệp vụ: hết hàng, đơn đã hủy, sửa dữ liệu không thuộc quyền.
- Kiểm thử giao diện: thông báo lỗi, giữ dữ liệu hợp lệ, phân trang.
- Kiểm thử API: JSON đúng cấu trúc, không lộ dữ liệu nhạy cảm.

8.6 Triển khai và cấu hình môi trường

- Tách cấu hình database khỏi mã nguồn nếu có thể.
- Không upload file chứa mật khẩu thật lên repository công khai.
- Kiểm tra đường dẫn public/index.php và quyền truy cập thư mục.
- Import schema.sql và seed.sql trước khi chạy thử.
- Viết README đủ rõ để người khác cài được project.

Bài 8 — Bước 7: Tạo endpoint JSON cho tìm sản phẩm

Mục tiêu: cùng dữ liệu từ Repository nhưng response là JSON thay vì HTML



Controller trả JSON tối giản

```
header('Content-Type: application/json; charset=utf-8');
$keyword = trim($_GET['q'] ?? '');
$data = $products->searchActive($keyword);

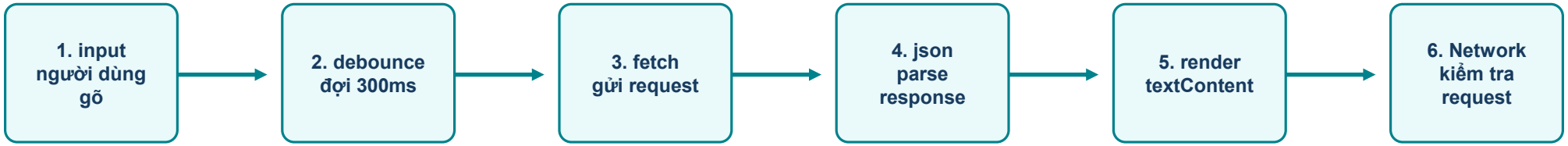
echo json_encode([
    'ok' => true,
    'data' => $data,
], JSON_UNESCAPED_UNICODE);
```

Nguyên tắc dữ liệu trả về

- Chỉ trả dữ liệu cần hiển thị: id, code, name, price, stock, category_name.
- Không trả password_hash, role nội bộ hoặc thông tin nhạy cảm.
- Nếu lỗi server: trả HTTP 500 + JSON có message rõ.
- Kiểm tra endpoint trực tiếp trên trình duyệt trước khi viết JavaScript.

Bài 8 — Bước 8: Gọi API bằng Fetch và cập nhật DOM an toàn

Mục tiêu: tìm kiếm sản phẩm không tải lại trang, có loading/error và không dùng innerHTML với dữ liệu chưa kiểm soát




```
const res = await fetch(
  'index.php?route=api/products&q=' + encodeURIComponent(keyword),
  { headers: { 'Accept': 'application/json' } }
);

if (!res.ok) throw new Error('HTTP ' + res.status);
const json = await res.json();

const td = document.createElement('td');
td.textContent = product.name;   // an toàn hơn innerHTML
```

Kiểm tra	Kết quả mong đợi
Từ khóa rỗng	trả tối đa 10 sản phẩm active
q=USB	chỉ thấy sản phẩm liên quan USB
Tắt database	hiển thị lỗi, không vỡ giao diện
Tên có <script>	không thực thi mã JS

DevTools → Network: sinh viên phải chỉ ra request, status code, response JSON.

Bài 8

Thực hành trên sản phẩm: live search sản phẩm

Áp dụng MiniShop hoặc đề tài nhóm

Bài tập tại slide

Tạo ô tìm kiếm sản phẩm; khi nhập từ khóa, gọi endpoint JSON và render danh sách sản phẩm cùng tồn kho.

- Không tải lại trang.
- Dữ liệu đúng CSDL.
- Không lỗi với từ khóa rỗng.
- Không dùng innerHTML với dữ liệu chưa kiểm soát.

Bài tập về nhà 8: tiếp nối từ buổi trước

Cá nhân	Nhóm	Yêu cầu nộp
• Hoàn thiện một endpoint JSON trong project cá nhân. • Viết giao diện gọi Fetch và hiển thị lỗi/loading. • Chuẩn bị checklist kiểm thử chức năng mình làm.	• Hoàn thiện bản release candidate bài tập lớn. • Chạy toàn bộ test case: form, CSDL, quyền, nghiệp vụ, API. • Cập nhật README hướng dẫn cài đặt và tài khoản test. • Chuẩn bị slide báo cáo và kịch bản demo 10 phút.	• Link repository bản release. • README hoàn chỉnh. • Checklist test có kết quả. • Slide báo cáo nhóm.

Bài 9

Báo cáo bài tập lớn

Mục tiêu và sản phẩm cuối bài

Trình bày bài toán, vai trò người dùng và phạm vi chức năng.

Giải thích ERD, schema.sql và các ràng buộc nghiệp vụ quan trọng.

Demo luồng chính trên dữ liệu mẫu.

Trả lời câu hỏi về code, CSDL, bảo mật và phân công.

9.1 Chuẩn bị trước ngày báo cáo

Pull code mới nhất và chạy lại project trên máy demo.

Import lại schema.sql và seed.sql trên CSDL sạch.

Chuẩn bị tài khoản test cho từng vai trò.

Kiểm tra luồng demo chính không phụ thuộc dữ liệu cũ trên máy cá nhân.

Mỗi thành viên nắm được phần mình làm và ít nhất một luồng end-to-end.

9.2 Kịch bản demo mẫu 20 phút

1 phút: bối cảnh bài toán và vai trò người dùng.

2 phút: ERD, bảng chính, ràng buộc quan trọng.

4 phút: demo luồng chính từ đăng nhập đến thao tác dữ liệu.

2 phút: điểm kỹ thuật nổi bật: PDO, MVC, phân quyền, Fetch/API.

1 phút: hạn chế và hướng hoàn thiện.

2 phút hỏi đáp cho mỗi thành viên (2x5=10 phút)

9.3 Rubric hỏi đáp kỹ thuật

Sinh viên giải thích được bảng và quan hệ trong CSDL.

Sinh viên chỉ ra được file xử lý request, file truy vấn CSDL và file giao diện.

Sinh viên chứng minh prepared statement được dùng đúng chỗ.

Sinh viên giải thích được kiểm tra quyền ở server.

Sinh viên trình bày được ít nhất một lỗi đã gặp và cách sửa.